

How do I Lean?

An amateur's guide to LLM-assisted formalisation

Sascha Troscheit

28 September 2026

EMU Seminar, Uppsala University



UPPSALA
UNIVERSITET

Lean

- Lean is a programming language whose type checker is a proof checker.



Lean

- Lean is a programming language whose type checker is a proof checker.
- A theorem is a type. A proof is a term of that type, which the kernel accepts or rejects.



Lean

- Lean is a programming language whose type checker is a proof checker.
- A theorem is a type. A proof is a term of that type, which the kernel accepts or rejects.
- Mathlib is the community library of definitions and theorems.



Lean

- Lean is a programming language whose type checker is a proof checker.
- A theorem is a type. A proof is a term of that type, which the kernel accepts or rejects.
- Mathlib is the community library of definitions and theorems.

```
theorem my_first_theorem (n :  $\mathbb{N}$ ) (h :  $3 < n$ ) :  $n \neq 0$  := by  
  sorry
```

Lean

- Lean is a programming language whose type checker is a proof checker.
- A theorem is a type. A proof is a term of that type, which the kernel accepts or rejects.
- Mathlib is the community library of definitions and theorems.

```
theorem my_first_theorem (n : ℕ) (h : 3 < n) : n ≠ 0 := by  
  sorry
```

sorry is a placeholder for a missing proof. Lean reports every **sorry** in a file. A project without any is **sorry-free**.



Lean

- Lean is a programming language whose type checker is a proof checker.
- A theorem is a type. A proof is a term of that type, which the kernel accepts or rejects.
- Mathlib is the community library of definitions and theorems.

```
theorem my_first_theorem (n : ℕ) (h : 3 < n) : n ≠ 0 := by  
  sorry
```

sorry is a placeholder for a missing proof. Lean reports every **sorry** in a file. A project without any is **sorry-free**.

LIMITATION

The kernel checks the proof. It does not check that the statement expresses the intended mathematics. Lean can (and does) have bugs.



History

- First tried in 2022. Went through *Mathematics in Lean*.



History

- First tried in 2022. Went through *Mathematics in Lean*.
- Gave up due to difficulties in proving statements.



History

- First tried in 2022. Went through *Mathematics in Lean*.
- Gave up due to difficulties in proving statements.
- Early 2026: alethfeld, vibefeld. Experiments in 'pseudo' formalisation using LLMs



History

- First tried in 2022. Went through *Mathematics in Lean*.
- Gave up due to difficulties in proving statements.
- Early 2026: alethfeld, vibefeld. Experiments in 'pseudo' formalisation using LLMs
- May 2026 onwards: Using LLM agents to write Lean proofs

History

- First tried in 2022. Went through *Mathematics in Lean*.
- Gave up due to difficulties in proving statements.
- Early 2026: alethfeld, vibefeld. Experiments in 'pseudo' formalisation using LLMs
- May 2026 onwards: Using LLM agents to write Lean proofs

Projects:

Project	Mathematics	Lines	Theorems	Audited
Galton–Watson trees	quasi-isometry classification	190 000	7 900	13
Brownian images	this talk	40 000	1 600	4 of 92
Fractal percolation	Assouad dimension on the $M \times N$ grid	38 000	1 500	5
GMT lecture notes	Sections 1–8 of the notes	19 000	600	19
Branching processes	Harris decomposition	8 000	360	7
Fractal geometry	Hutchinson, Ahlfors regularity	3 000	210	6

Before Lean: adversarial proof trees

- Alethfeld and Vibefeld (T. Osborne, 2025–26): natural-language proofs by AI agents in separated roles, without a formal kernel.



Before Lean: adversarial proof trees

- Alethfeld and Vibefeld (T. Osborne, 2025–26): natural-language proofs by AI agents in separated roles, without a formal kernel.
- The proof is a tree of numbered claims (1, 1.1, 1.1.1, ...). A prover agent refines a claim into steps; a verifier agent challenges a step; the prover refines further, or the step is rejected.

Before Lean: adversarial proof trees

- Alethfeld and Vibefeld (T. Osborne, 2025–26): natural-language proofs by AI agents in separated roles, without a formal kernel.
- The proof is a tree of numbered claims (1, 1.1, 1.1.1, ...). A prover agent refines a claim into steps; a verifier agent challenges a step; the prover refines further, or the step is rejected.
- Every action is appended to a ledger with hashes.

Before Lean: adversarial proof trees

- Alethfeld and Vibefeld (T. Osborne, 2025–26): natural-language proofs by AI agents in separated roles, without a formal kernel.
- The proof is a tree of numbered claims (1, 1.1, 1.1.1, ...). A prover agent refines a claim into steps; a verifier agent challenges a step; the prover refines further, or the step is rejected.
- Every action is appended to a ledger with hashes.

RESULT

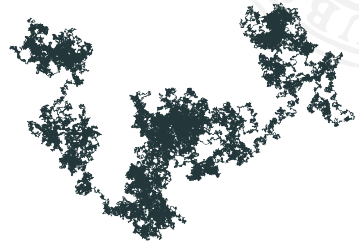
This gives a structured argument with gaps visible. The formal proof came later, the vibefeld tree was its plan.

A 'real live' example



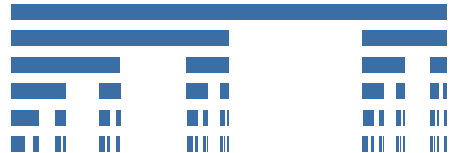
An inverse Problem for Brownian motion

- Planar Brownian motion W : a particle with independent random increments.



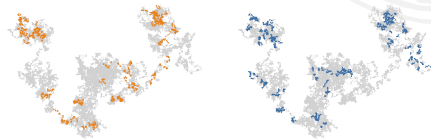
An inverse Problem for Brownian motion

- Planar Brownian motion W : a particle with independent random increments.
- A probability measure μ on the time interval $[0, 1]$. Two examples, μ_A and μ_B , supported on Cantor sets of the same dimension.



An inverse Problem for Brownian motion

- Planar Brownian motion W : a particle with independent random increments.
- A probability measure μ on the time interval $[0, 1]$. Two examples, μ_A and μ_B , supported on Cantor sets of the same dimension.
- The image measure $\nu = W_*\mu$: the positions of the path at μ -distributed times.

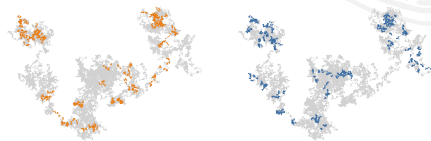


An inverse Problem for Brownian motion

- Planar Brownian motion W : a particle with independent random increments.
- A probability measure μ on the time interval $[0, 1]$. Two examples, μ_A and μ_B , supported on Cantor sets of the same dimension.
- The image measure $\nu = W_*\mu$: the positions of the path at μ -distributed times.

QUESTION (M. HOCHMAN, 2022)

Given one sample of ν , was it produced by μ_A or by μ_B ?



Distinguishing the two measures



- The dimension of the image measure is almost surely the same for μ_A and μ_B .

Distinguishing the two measures



- The dimension of the image measure is almost surely the same for μ_A and μ_B .
- The correlation profile differs: the rescaled probability that two independent points of ν lie within distance r , as a function of r .

Distinguishing the two measures



- The dimension of the image measure is almost surely the same for μ_A and μ_B .
- The correlation profile differs: the rescaled probability that two independent points of ν lie within distance r , as a function of r .
- **Theorem** (T., 2026). If the expected correlation profiles of μ_A and μ_B remain separated, the laws of the two image measures are mutually singular. One sample almost surely determines its source.

The theorem in Lean (Challenge.lean, comments added)

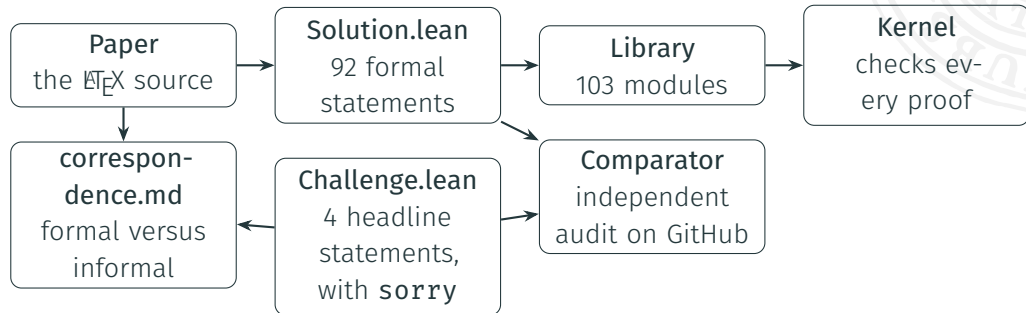
```
theorem audit_main
  -- a probability space and a planar Brownian motion on it
  {P : Measure  $\Omega$ } [IsProbabilityMeasure P]
  {W :  $\mathbb{R}_{\geq 0} \rightarrow \Omega \rightarrow \text{Plane}$ } (hW : IsPlanarBrownian W P)
  -- two Frostman probability measures on the line
  {s A1 A2 :  $\mathbb{R}$ } (hs0 : 0 < s) (hs1 : s < 1)
  { $\mu_1$   $\mu_2$  : Measure  $\mathbb{R}$ } [IsProbabilityMeasure  $\mu_1$ ] [IsProbabilityMeasure  $\mu_2$ ]
  (h1 : IsFrostman s A1  $\mu_1$ ) (h2 : IsFrostman s A2  $\mu_2$ )
  -- whose expected correlation profiles remain separated
  (hsep :  $\exists \varepsilon > 0, \forall V : \mathbb{R}, \exists t \geq V,$ 
     $\varepsilon \leq |\text{expectedProfile } W \text{ } P \text{ } s \text{ } \mu_1 \text{ } t - \text{expectedProfile } W \text{ } P \text{ } s \text{ } \mu_2 \text{ } t|$ ) :
  -- then the laws of the image measures are mutually singular
  (occupationLaw W P  $\mu_1$ ).MutuallySingular (occupationLaw W P  $\mu_2$ )
```

The theorem in Lean (`Challenge.lean`, comments added)

```
theorem audit_main
  -- a probability space and a planar Brownian motion on it
  {P : Measure  $\Omega$ } [IsProbabilityMeasure P]
  {W :  $\mathbb{R}_{\geq 0} \rightarrow \Omega \rightarrow \text{Plane}$ } (hW : IsPlanarBrownian W P)
  -- two Frostman probability measures on the line
  {s A1 A2 :  $\mathbb{R}$ } (hs0 : 0 < s) (hs1 : s < 1)
  { $\mu_1$   $\mu_2$  : Measure  $\mathbb{R}$ } [IsProbabilityMeasure  $\mu_1$ ] [IsProbabilityMeasure  $\mu_2$ ]
  (h1 : IsFrostman s A1  $\mu_1$ ) (h2 : IsFrostman s A2  $\mu_2$ )
  -- whose expected correlation profiles remain separated
  (hsep :  $\exists \varepsilon > 0, \forall V : \mathbb{R}, \exists t \geq V,$ 
     $\varepsilon \leq |\text{expectedProfile } W \ P \ s \ \mu_1 \ t - \text{expectedProfile } W \ P \ s \ \mu_2 \ t|$ ) :
  -- then the laws of the image measures are mutually singular
  (occupationLaw W P  $\mu_1$ ).MutuallySingular (occupationLaw W P  $\mu_2$ )
```

Hypotheses before the final colon, conclusion after it.

Organisation of a project



Cautionary tale



July 2026: a disproof of the Collatz conjecture

- 25 July: a sorry-free Lean disproof, produced with AI assistance, is published.
No axioms are reported, and the independent checker nanoda accepts it.

July 2026: a disproof of the Collatz conjecture

- 25 July: a sorry-free Lean disproof, produced with AI assistance, is published. No axioms are reported, and the independent checker nanoda accepts it.
- 28 July: K. Gopinathan reduces it to a proof of **False**: a metaprogram had sent an ill-typed declaration to the kernel directly, and nanoda had a second bug. Fixed within an hour.

July 2026: a disproof of the Collatz conjecture

- 25 July: a sorry-free Lean disproof, produced with AI assistance, is published. No axioms are reported, and the independent checker nanoda accepts it.
- 28 July: K. Gopinathan reduces it to a proof of **False**: a metaprogram had sent an ill-typed declaration to the kernel directly, and nanoda had a second bug. Fixed within an hour.
- 30 July to 20 August: a bug hunt with an OpenAI model finds about ten more bugs.

July 2026: a disproof of the Collatz conjecture

- 25 July: a sorry-free Lean disproof, produced with AI assistance, is published. No axioms are reported, and the independent checker nanoda accepts it.
- 28 July: K. Gopinathan reduces it to a proof of **False**: a metaprogram had sent an ill-typed declaration to the kernel directly, and nanoda had a second bug. Fixed within an hour.
- 30 July to 20 August: a bug hunt with an OpenAI model finds about ten more bugs.
- L. de Moura: “We consider AI-generated proofs a potential source of malicious proofs.”

July 2026: a disproof of the Collatz conjecture

- 25 July: a sorry-free Lean disproof, produced with AI assistance, is published. No axioms are reported, and the independent checker nanoda accepts it.
- 28 July: K. Gopinathan reduces it to a proof of **False**: a metaprogram had sent an ill-typed declaration to the kernel directly, and nanoda had a second bug. Fixed within an hour.
- 30 July to 20 August: a bug hunt with an OpenAI model finds about ten more bugs.
- L. de Moura: “We consider AI-generated proofs a potential source of malicious proofs.”

Agents learn the same shortcuts. Trained to close goals, they find renamed **sorry**, extra axioms, **native_decide**, weakened statements, altered definitions, and kernel exploits. They may corrupt the local Lean installation.

Comparator

- Input: **Challenge.lean**, the statements with **sorry** over Mathlib only;
Solution.lean, the same statements with proofs; a configuration naming the theorems and the permitted axioms.

Comparator

- Input: **Challenge.lean**, the statements with **sorry** over Mathlib only; **Solution.lean**, the same statements with proofs; a configuration naming the theorems and the permitted axioms.
- Checks:
 1. the statements of Challenge, with every definition they depend on, are identical in Solution;
 2. the proofs use only the permitted axioms;
 3. the Solution environment is exported and replayed through the Lean kernel and, optionally, through independent checkers (nanoda, lean4lean), in a sandbox.

Comparator

- Input: **Challenge.lean**, the statements with **sorry** over Mathlib only; **Solution.lean**, the same statements with proofs; a configuration naming the theorems and the permitted axioms.
- Checks:
 1. the statements of Challenge, with every definition they depend on, are identical in Solution;
 2. the proofs use only the permitted axioms;
 3. the Solution environment is exported and replayed through the Lean kernel and, optionally, through independent checkers (nanoda, lean4lean), in a sandbox.
- Not checked: whether **Challenge.lean** states the right theorem. Its definitions are read by a human.

What is checked

CHECKED BY THE MACHINE

- every proof step, down to three standard axioms
- the public statements agree with the proved ones
- independent checkers replay the proofs on every push

CHECKED BY THE AUTHOR

- that the formal statement is the theorem of the paper
- that the definitions, such as **IsFrostman**, are the standard notions
- that the theorem is of interest

What is checked

CHECKED BY THE MACHINE

- every proof step, down to three standard axioms
- the public statements agree with the proved ones
- independent checkers replay the proofs on every push

CHECKED BY THE AUTHOR

- that the formal statement is the theorem of the paper
- that the definitions, such as **IsFrostman**, are the standard notions
- that the theorem is of interest

The correspondence document lists ways in which the formal statement could deviate from the paper. The comparator compares **Challenge.lean** with **Solution.lean**, never with the paper: the correspondence remains a human obligation.

Tau Ceti: a shared library beside Mathlib

WHAT IT IS

A library of formal mathematics on top of Mathlib. Humans write the roadmap; AI agents write and review the code. Lean FRO and Mathlib Initiative, 2026.

Tau Ceti: a shared library beside Mathlib

WHAT IT IS

A library of formal mathematics on top of Mathlib. Humans write the roadmap; AI agents write and review the code. Lean FRO and Mathlib Initiative, 2026.

Why. Formalisations are duplicated: projects pin versions, copy each other's code, and prove the same results again.

Tau Ceti: a shared library beside Mathlib

WHAT IT IS

A library of formal mathematics on top of Mathlib. Humans write the roadmap; AI agents write and review the code. Lean FRO and Mathlib Initiative, 2026.

Why. Formalisations are duplicated: projects pin versions, copy each other's code, and prove the same results again.

How it works.

- Humans decide what is formalised, in a roadmap reviewed by humans.
- AI agents write the code and review it against a public rubric.
- A registry of intentions: no two contributors take the same item.

Tau Ceti: a shared library beside Mathlib

WHAT IT IS

A library of formal mathematics on top of Mathlib. Humans write the roadmap; AI agents write and review the code. Lean FRO and Mathlib Initiative, 2026.

Why. Formalisations are duplicated: projects pin versions, copy each other's code, and prove the same results again.

How it works.

- Humans decide what is formalised, in a roadmap reviewed by humans.
- AI agents write the code and review it against a public rubric.
- A registry of intentions: no two contributors take the same item.

Relation to Mathlib. Built on Mathlib, never merged into it. A temporary solution.

Palomar: a registry of machine-checked results

WHAT IT IS

A public, searchable list of Lean-verified theorems. Each entry is a formal statement at a fixed commit whose proof has been checked. Lean FRO and ICARM, 2026.

Palomar: a registry of machine-checked results

WHAT IT IS

A public, searchable list of Lean-verified theorems. Each entry is a formal statement at a fixed commit whose proof has been checked. Lean FRO and ICARM, 2026.

An entry contains

- the statement, the repository, and the exact commit;
- a comparator check with two independent kernels;
- an LLM opinion on whether the formal statement matches the informal claim;
- a declaration of which AI tools were used.

Palomar: a registry of machine-checked results

WHAT IT IS

A public, searchable list of Lean-verified theorems. Each entry is a formal statement at a fixed commit whose proof has been checked. Lean FRO and ICARM, 2026.

An entry contains

- the statement, the repository, and the exact commit;
- a comparator check with two independent kernels;
- an LLM opinion on whether the formal statement matches the informal claim;
- a declaration of which AI tools were used.

What it is not. Not peer review, not a journal, not a certificate of novelty.

Lessons

- Most of the work is in the $\text{\LaTeX}$.



Lessons



- Most of the work is in the $\text{\LaTeX}$.
- The definitions determine what a theorem means. They need more scrutiny than the proofs.

Lessons



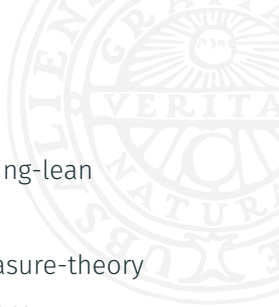
- Most of the work is in the $\text{\LaTeX}$.
- The definitions determine what a theorem means. They need more scrutiny than the proofs.
- Reading Lean statements is necessary. Writing Lean proofs maybe not.

Lessons



- Most of the work is in the $\text{\LaTeX}$.
- The definitions determine what a theorem means. They need more scrutiny than the proofs.
- Reading Lean statements is necessary. Writing Lean proofs maybe not.
- Inefficiencies with multiple developments. A collective approach is needed.

Links



Galton–Watson trees	github.com/sascha2718/qi-trees-and-matching-lean
Tree paper	arXiv:2609.23882
GMT lecture notes	github.com/sascha2718/lean-geometric-measure-theory
Mathlib	leanprover-community.github.io/mathlib4_docs
Search	loogle.lean-lang.org , leansearch.net
Vibefeld	github.com/tobiasosborne/vibefeld
Comparator	github.com/leanprover/comparator
Tau Ceti, Palomar	github.com/TauCetiProject/TauCeti , palomar-registry.org
Kernel bug postmortems	leodemoura.github.io/blog , 1 August 2026, 24 August 2026